Dynamic QR Code Integration Guide

Version: 2.0

Table of Contents

1. [Overview](#1-overview)
2. [Environments & Base URLs](#2-environments--base-urls)
3. [Authentication & Encryption](#3-authentication--encryption)
4. [API Reference](#4-api-reference)
   - 4.1 [Generate Dynamic QR Code](#41-generate-dynamic-qr-code)
   - 4.2 [Check Transaction Status](#42-check-transaction-status)
   - 4.3 [Transaction Callback (MBK -> Partner)](#43-transaction-callback-mbk---partner)
   - 4.4 [Cancel Transaction](#44-cancel-transaction)
   - 4.5 [Perform Mock Transaction (Test Only)](#45-perform-mock-transaction-test-only)
5. [Status Codes Reference](#5-status-codes-reference)
6. [Encryption & Decryption](#6-encryption--decryption)
   - 6.1 [Key Generation](#61-key-generation)
   - 6.2 [Encryption Flow](#62-encryption-flow)
   - 6.3 [Decryption Flow](#63-decryption-flow)
   - 6.4 [Sample Java Implementation](#64-sample-java-implementation)

1. Overview

Dynamic QR Code APIs enable integrating partners to offer dynamic QR payment services to their merchants (and sub-merchants). These APIs support the complete transaction lifecycle, including:

- **Generate** a UPI-compatible dynamic QR code with a predefined transaction amount
- **Fetch** real-time status of QR-based payments
- **Receive** asynchronous notifications upon successful transactions
- **Refund** a completed transaction or **Cancel** an upcoming transaction
- **Trigger** mock transactions for testing and integration validation

Two endpoint prefixes are available:

Prefix	Purpose	Content-Type	Encryption
/merchant/testdqr	UAT / Testing -- plain JSON for easy debugging	application/json	None
/merchant/dqr	UAT & Production -- encrypted payload	text/plain	RSA + AES

> *Integration tip: Start with /testdqr during development, then switch to /dqr once encryption is implemented.*

2. Environments & Base URLs

Placeholder	Description
<MBK_HOST_URL>	MobiKwik base URL (provided during onboarding)
<PARTNER_HOST_URL>	Partner's base URL (provided by integrating partner)

All API paths in this document are relative to `<MBK_HOST_URL>/walletapis`.

3. Authentication & Encryption

3.1 Test Endpoints (`/testdqr`)

Header	Required	Description
clientId	Yes	Unique client identifier assigned during onboarding

Request and response bodies are **plain JSON** -- no encryption.

3.2 Production Endpoints (`/dqr`)

Header	Required	Description
clientId	Yes	Unique client identifier assigned during onboarding
key	Yes	RSA-encrypted AES key (Base64-encoded)

Request and response bodies are **encrypted** -- see [Section 6](#6-encryption--decryption) for full details.

4. API Reference

4.1 Generate Dynamic QR Code

Creates a new dynamic QR code for a given amount.

Endpoints

Variant	Method	URL
Test	POST	<MBK_HOST_URL>/walletapis/merchant/testdqr/generate
Encrypted	POST	<MBK_HOST_URL>/walletapis/merchant/dqr/generate

Request Headers

Test (`/testdqr`)

Header	Value
Content-Type	application/json
clientId	<your_client_id>

Encrypted (`/dqr`)

Header	Value
Content-Type	text/plain
clientId	<your_client_id>
key	<RSA_encrypted_AES_key>

Request Body (Plain JSON)

```
{
  "orderId": "5fbe5c285c9791684339113183",
  "amount": 100.00,
  "mid": "MBK123",
  "smid": "SMID123",
  "imageRequired": true
}
```

Encrypted Request Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =qdnw2523j23jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi*#$*
sdmcow3092m35n45l43634128jnjlkn12oun1m2eo1j9eh12ej0238uej238r9eu2j3
ej29ije2o3e2i3uje9082u3ejowje1
```

> *Note: Request body is encrypted by a random generated AES key. The same key is passed in the header, encrypted using MBK's RSA public key.*

Request Fields

Field	Type	Required	Description
orderId	string	No	Unique order ID (12-50 characters). Strongly recommended, but auto-generated by MBK if not provided.
amount	number	Yes	Amount in INR (min: 1.00, max: 200000.00). Supports up to 2 decimal places.
mid	string	Yes	Unique parent merchant ID (3-12 characters).
smid	string	No	Unique child/sub-merchant ID (4-12 characters).
imageRequired	boolean	No	Set true to receive QR image as Base64 string. Default: false.

Response Body (Plain JSON)

```
{
  "statusCode": 0,
  "statusMessage": "Waiting for payment",
  "orderId": "5fbe5c285c9791684339113183",
  "qrData": "upi://pay?pa=merchant@bank&pn=StoreName&amt=100&mc=5499&mode=01",
  "image": "<Base64_encoded_QR_image_or_null>"
}
```

Encrypted Response Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =13k4of9jsdn3l28usdns9$j2kjd(bsbW^heuk329dhwnd298hHyu@KNJIDUnI
JDU23e23n(dKJDnk4kb2K$B@IOU*DWoijsLNiojoiwjIJI@()JDW()js0d82j2109u32
```

Note: Response body is encrypted by MBK with a random AES key. The key is returned in the response header, encrypted using the partner's RSA public key. Decrypt the key with your private key, then decrypt the body.

Response Fields

Field	Type	Always Present	Description
statusCode	integer	Yes	Status code (see [Section 5](#5-status-codes-reference)).
statusMessage	string	Yes	Human-readable status message.
orderId	string	Yes	Order ID (generated by MBK if not provided in request).
qrData	string	Yes	UPI intent URL to encode into a QR code.
image	string	No	Base64-encoded QR image (only when imageRequired: true).

4.2 Check Transaction Status

Retrieves the current status of a transaction initiated via dynamic QR code.

Endpoints

Variant	Method	URL
Test	POST	<MBK_HOST_URL>/walletapis/merchant/testdqr/txnstatus
Encrypted	POST	<MBK_HOST_URL>/walletapis/merchant/dqr/txnstatus

Request Headers

Test (`/testdqr`)

Header	Value
Content-Type	application/json
clientId	<your_client_id>

Encrypted (`/dqr`)

Header	Value
Content-Type	text/plain
clientId	<your_client_id>
key	<RSA_encrypted_AES_key>

Request Body (Plain JSON)

```
{
  "orderId": "5fbe5c285c9791684339113183",
  "mid": "MBK123",
  "smid": "SMBK123"
}
```

Encrypted Request Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =!K#OI*YFNJNJDNOWIO!87983e23ejl)(5bHGGw2782hg2jjuJ(S&ghi3e289HJ
H7yhKB7HGjhb037212008460069
```

Request Fields

Field	Type	Required	Description
orderId	string	Yes	Order ID of the transaction to check.
mid	string	Yes	Unique parent merchant ID.
smid	string	No	Child/sub-merchant ID.

Response Body -- Success (Plain JSON)

```
{
  "statusCode": 100,
  "statusMessage": "Payment successful from bank",
  "orderId": "5fbe5c285c9791684339113183",
  "bankRRN": "09123701283"
}
```

Response Body -- Refunded (Plain JSON)

```
{
  "statusCode": 1002,
  "statusMessage": "Order is refunded",
  "orderId": "5fbe5c285c9791684339113183",
  "bankRRN": "09123701283",
  "refundId": "REF123456789012"
}
```

Encrypted Response Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =a1309uklnelopjkdw8sd6Hhsa687KBH78adyhjbndsdoiu&^654#)*nlknscals
caksnclkascl2e087129u
```

Response Fields

Field	Type	Always Present	Description
statusCode	integer	Yes	Status code (see [Section 5](#5-status-codes-reference)).
statusMessage	string	Yes	Human-readable status message.
orderId	string	No	Order ID of the transaction.
bankRRN	string	No	Bank reference number (present on payment success).
refundId	string	No	Refund reference ID (present if order was refunded).

4.3 Transaction Callback (MBK -> Partner)

MBK sends a callback to the partner's server upon **successful** payment. The callback host URL is to be provided by the integrating partner, and the callback response should be fixed as below.

> *Note: Callbacks are sent only for successful transactions.*

Endpoint

Method	URL
POST	<PARTNER_HOST_URL>/dynamicqr/transaction/callback

Request Headers

Header	Value
Content-Type	application/json or text/plain
key	<RSA_encrypted_AES_key> (for encrypted mode)

Callback Body (Decrypted)

```
{
  "mid": "MBK0291670",
  "merchantName": "HOLY_FAMILY_HOSPITAL-POPD",
  "smid": "SMBK34729",
  "userVPA": "abhipriya1@ptyes",
  "userCell": "NA",
  "merchantVPA": "ombk.aach8232121lqm188yr1@mbk",
  "transactionId": "20260429029167020fd2d8ed30c9",
  "transactionAmount": 1200.00,
  "refundAmount": 0,
  "transactionType": "PAYMENT",
  "transactionStatus": "SUCCESS",
  "currencyType": "INR",
  "transactionTime": "Apr 29, 2026, 3:56:25 PM",
  "bankRRN": "603646624741"
}
```

Encrypted Callback Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =!K#OI*YFNJNJDNOWIO!87983e23ejl)(5bHGGw2782hg2jjuJ(S&ghi3e289HJ
H7yhKB7HGjhb
```

Callback Fields

Field	Type	Description
mid	string	Parent merchant ID.
merchantName	string	Merchant display name.
smid	string	Sub-merchant ID (if applicable).
userVPA	string	Payer's UPI VPA.
userCell	string	Payer's mobile number (or "NA" if unavailable).
merchantVPA	string	Merchant's UPI VPA used for collection.
transactionId	string	Unique transaction / order ID.
transactionAmount	number	Transaction amount in INR.
refundAmount	number	Refund amount (0 if no refund).
transactionType	string	Type of transaction (e.g. "PAYMENT").
transactionStatus	string	Status of the transaction (e.g. "SUCCESS").

Field	Type	Description
currencyType	string	Currency code (e.g. "INR").
transactionTime	string	Timestamp of the transaction.
bankRRN	string	Bank reference number.

4.4 Cancel Transaction

Initiates a cancellation/refund for a transaction done via dynamic QR code. If a cancellation has already been initiated, this API returns the latest status.

> *Note: Partial refund is only allowed when the transaction is successful and a callback was provided. Otherwise, refundAmount will be ignored in case of cancellation.*

Endpoints

Variant	Method	URL
Test	POST	<MBK_HOST_URL>/walletapis/merchant/testdqr/cancel
Encrypted	POST	<MBK_HOST_URL>/walletapis/merchant/dqr/cancel

Request Headers

Test (`/testdqr`)

Header	Value
Content-Type	application/json
clientId	<your_client_id>

Encrypted (`/dqr`)

Header	Value
Content-Type	text/plain
clientId	<your_client_id>
key	<RSA_encrypted_AES_key>

Request Body (Plain JSON)

```
{
  "refundAmount": 5.00,
  "orderId": "5fbe5c285c9791684339113183",
  "mid": "MBK123",
  "smid": "SMBK234",
  "refundRefId": "REFUND20250628001"
}
```

Encrypted Request Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =!K#OI*YFNJNJDNOWIO!87983e23ejl)(5bHGGw2782hg2jjuJ(S&ghi3e289HJ
H7yhKB7HGjhb
```

Request Fields

Field	Type	Required	Description
orderId	string	Yes	Order ID of the original transaction (12-50 characters).
mid	string	Yes	Unique parent merchant ID (3-12 characters).
smid	string	No	Sub-merchant ID (4-12 characters).
refundAmount	number	Yes	Amount in INR to refund (min: 1.00, max: 200000.00).
refundRefId	string	Yes	Unique refund reference ID for this attempt (12-50 characters).

Response Body (Plain JSON)

```
{
  "statusCode": "0",
  "statusMessage": "Refund Initiated"
}
```

Encrypted Response Body (Sample)

```
Header key: =qdnw25sa9u2n3jn23jnr2o3jr289y8ywsdwqd1=#o48i()2km4n5bijdfi
*#$*sdmcow3092m35n45l436341281eh12ej0238uej238r9eu2j3ej29ije2o3e2i3u
je9082u3ejowje1=@*D*D&F9

Body: =a1309uklnelopjkdw8sd6Hhsa687KBH78adyhjbndsdoiu&^654#)*nlknscals
caksnclkascl2e087129u
```

Response Fields

Field	Type	Always Present	Description
statusCode	string	Yes	Status code (see [Section 5](#5-status-codes-reference)).
statusMessage	string	Yes	Human-readable status message.

4.5 Perform Mock Transaction (Test Only)

Simulates a successful payment against an order created via the Generate Dynamic QR Code API. This is available **only on the test endpoint** (`/testdqr`) and is intended for UAT integration testing.

> *Note: This API is for testing purposes only and is not available in production.*

Sample cURL

```
curl --location --request POST \
  '<MBK_HOST_URL>/walletapis/merchant/testdqr/performMockTxn
    ?orderId=<orderID>&is3P=true' \
  --header 'Content-Type: application/json'
```

Request Parameters (Query Params)

Parameter	Type	Required	Description
orderId	string	Yes	Order ID of the DQR transaction to simulate payment for.
is3P	boolean	Yes	To mock a txn done via MobiKwik app, keep is3P=false. To mock txns done via other apps, keep is3P=true.

Response Body

```
{
  "success": true
}
```

5. Status Codes Reference

General Status Codes

Code	Meaning	Description
0	Pending	Waiting for payment / Refund initiated
100	Success	Payment is successful / Refund is successful

Client Errors (400-499) -- Fix request and retry

Code	Message
400	Client Error! Bad request
401	Client Error! Unauthorized access
402	Bad request, Invalid value of parameter X
403	Bad request, Access Forbidden
404	Bad request, Invalid Order Id / Order not found

Server Errors (500-599) -- Retry after some time

Code	Message
500	Internal Server Error
503	Service unavailable, Please try again

Order Failure Codes (1000+) -- Applicable only to Check Transaction Status API

Code	Message
1000	QR code expired. Any amount deducted against this order will be reversed.
1001	Payment could not be completed. Any amount deducted against this order will be reversed.
1002	Order is refunded.

Status Code Decision Guide

```
statusCode == 0         -> Transaction is pending / Refund initiated. Poll again.
statusCode == 100       -> Success. Transaction complete / Refund complete.
400 <= statusCode < 500 -> Client error. Fix the request and retry.
500 <= statusCode < 600 -> Server error. Retry after a short delay.
statusCode >= 1000      -> Terminal failure (only in Check Status). Order is final.
```

6. Encryption & Decryption

All requests and responses on the `/dqr` endpoints are encrypted using a **hybrid RSA + AES** scheme.

6.1 Key Generation

Generate a 4096-bit RSA key pair:

```
# Generate RSA private key
openssl genrsa -out private_key.pem 4096

# Convert to PKCS#8 DER format (for Java)
openssl pkcs8 -topk8 -nocrypt -in private_key.pem -outform der -out private.der

# Export public key in DER format (for Java)
openssl rsa -in private_key.pem -pubout -outform DER -out public_key.der
```

Both MBK and the partner generate their own key pairs and exchange **public keys**.

6.2 Encryption Flow (Sending a Request to MBK)

```
1. Generate a random AES key (e.g. "abcdefghijklmop")
2. Encrypt the JSON request body using AES/CBC/PKCS5Padding with the random key
3. Encrypt the random AES key using RSA/ECB/PKCS1Padding with MBK's public key
4. Send:
   - Header "key" = Base64(RSA-encrypted AES key)
   - Body = Base64(AES-encrypted JSON)
```

6.3 Decryption Flow (Receiving a Response from MBK)

```
1. Read the "key" header from the response
2. Decrypt the "key" using RSA/ECB/PKCS1Padding with your private key -> AES key
3. Decrypt the response body using AES/CBC/PKCS5Padding with the decrypted AES key -> JSON
```

6.4 Sample Java Implementation

Algorithms used:

- Asymmetric: `RSA/ECB/PKCS1Padding`
- Symmetric: `AES/CBC/PKCS5Padding`

```
import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.security.*;
import java.security.spec.PKCS8EncodedKeySpec;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;

public class EncryptionDecryptionV2Utils {

    private static final String RSA_ALGO = "RSA/ECB/PKCS1Padding";
    private static final String AES_ALGO = "AES/CBC/PKCS5Padding";

    public static String encryptRSA(String msg, String pubKeyPath) {
        try {
            KeyFactory kf = KeyFactory.getInstance("RSA");
            Cipher cipher = Cipher.getInstance(RSA_ALGO);
            byte[] pubBytes = Files.readAllBytes(new File(pubKeyPath).toPath());
            cipher.init(Cipher.ENCRYPT_MODE,
```

```
            kf.generatePublic(new X509EncodedKeySpec(pubBytes)));
        return Base64.getEncoder().encodeToString(
            cipher.doFinal(msg.getBytes(StandardCharsets.UTF_8)));
    } catch (Exception e) { e.printStackTrace(); return null; }
}

public static String decryptRSA(String encrypted, String privKeyPath) {
    try {
        KeyFactory kf = KeyFactory.getInstance("RSA");
        Cipher cipher = Cipher.getInstance(RSA_ALGO);
        byte[] privBytes = Files.readAllBytes(new File(privKeyPath).toPath());
        cipher.init(Cipher.DECRYPT_MODE,
            kf.generatePrivate(new PKCS8EncodedKeySpec(privBytes)));
        return new String(cipher.doFinal(
            Base64.getDecoder().decode(encrypted)), StandardCharsets.UTF_8);
    } catch (Exception e) { e.printStackTrace(); return null; }
}

public static String encryptAES(String plainText, String key) {
    try {
        byte[] iv = new byte[16];
        new SecureRandom().nextBytes(iv);
        IvParameterSpec ivSpec = new IvParameterSpec(iv);
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        digest.update(key.getBytes("UTF-8"));
        byte[] keyBytes = new byte[16];
        System.arraycopy(digest.digest(), 0, keyBytes, 0, 16);
        SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "AES");
        Cipher cipher = Cipher.getInstance(AES_ALGO);
        cipher.init(Cipher.ENCRYPT_MODE, secretKey, ivSpec);
        byte[] encrypted = cipher.doFinal(plainText.getBytes());
        byte[] combined = new byte[16 + encrypted.length];
        System.arraycopy(iv, 0, combined, 0, 16);
        System.arraycopy(encrypted, 0, combined, 16, encrypted.length);
        return Base64.getEncoder().encodeToString(combined);
    } catch (Exception e) { e.printStackTrace(); return null; }
}

public static String decryptAES(String encryptedMsg, String key) {
    try {
        byte[] data = Base64.getDecoder().decode(encryptedMsg);
        byte[] iv = new byte[16];
        System.arraycopy(data, 0, iv, 0, 16);
        int encSize = data.length - 16;
        byte[] encBytes = new byte[encSize];
        System.arraycopy(data, 16, encBytes, 0, encSize);
        byte[] keyBytes = new byte[16];
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        md.update(key.getBytes());
        System.arraycopy(md.digest(), 0, keyBytes, 0, 16);
        Cipher cipher = Cipher.getInstance(AES_ALGO);
        cipher.init(Cipher.DECRYPT_MODE,
            new SecretKeySpec(keyBytes, "AES"), new IvParameterSpec(iv));
        return new String(cipher.doFinal(encBytes));
    } catch (Exception e) { e.printStackTrace(); return null; }
}

public static void main(String[] args) {
    String key = "abcdefghijklmop";
    String json = "{\"orderId\":\"TEST123456789012\","
        + "\"amount\":100.00,\"mid\":\"MBK123\"}";

    // PARTNER SIDE: Sending request to MBK
    String aesEncMsg = encryptAES(json, key);
    String rsaEncKey = encryptRSA(key, "/path/to/mbk_public.der");
    // Send: Header "key" = rsaEncKey, Body = aesEncMsg
```

```
        // PARTNER SIDE: Receiving response from MBK
        String rsaDecKey = decryptRSA(rsaEncKey, "/path/to/partner_priv.der");
        String aesDecMsg = decryptAES(aesEncMsg, rsaDecKey);
        System.out.println("Decrypted: " + aesDecMsg);
    }
}
```

Quick Reference Card

API	Test Endpoint (`/testdqr`)	Production Endpoint (`/dqr`)
Generate QR	POST <MBK_HOST_URL>/walletapis/merchant/testd	POST <MBK_HOST_URL>/walletapis/merchant/dqr/g
Check Status	POST <MBK_HOST_URL>/walletapis/merchant/testd	POST <MBK_HOST_URL>/walletapis/merchant/dqr/tx
Cancel	POST <MBK_HOST_URL>/walletapis/merchant/testd	POST <MBK_HOST_URL>/walletapis/merchant/dqr/c
Mock Transaction	POST <MBK_HOST_URL>/walletapis/merchant/testd	--
Callback	POST <PARTNER_HOST_URL>/dynamicqr/transaction/callback	POST <PARTNER_HOST_URL>/dynamicqr/transaction/callback

Environment Setup

Parameter	UAT	Production
URL	To be shared by MBK	https://walletapi.mobikwik.com
MID/SMID	a new mid/smid to be generated and shared by MBK Tech	To be provided by MBK Business
clientId	a new clientId to be shared by MBK Tech	a new clientId to be setup and shared by MBK Tech
MBK Public Key	a new key to be generated and shared by MBK Tech	a new key to be generated and shared by MBK Tech
Partner Public Key	To be generated and shared by Partner	To be generated and shared by Partner

End of Document